Driving the IOTBT701 RGB strip from a Windows laptop: Ambilight over BLE

Analytical report, Oct 29 – Nov 2, 2026 (4 sessions) • github.com/Ne-collab228/iotbt701-ambilight • GUI v1.5

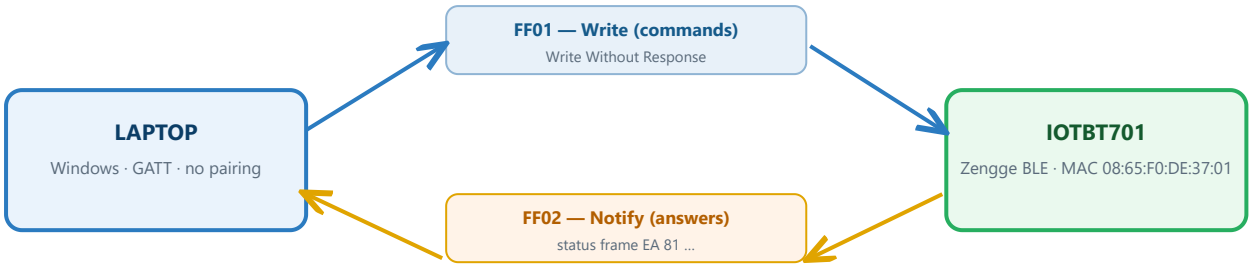
v1.5	**10**	**5**	**16.9/s**	**3**
GUI + startup toggle + log file + "LINK LOST" window	commits, MIT, public repo	documentation languages (EN/RU/ES/ZH/PT-BR)	sustained write rate (24.4-min soak)	"fake limits" removed by measurement

Goal: an **IOTBT701** RGB strip (Zengge/LEDnetWF BLE controller, MAC 08:65:F0:DE:37:01) mirrors the laptop's average screen colour. No phone app, no pairing, pure Win32/.NET. Result: a finished GUI product, a public repo with complete protocol docs, and a CLI toolset — plus every "limit" we had initially written down for ourselves was tested and honestly removed.

1. Timeline

Date	Milestone	What happened
Oct 29–30	**Protocol cracked**	BLE scan without pairing (Windows PairAsync always returns "Failed" — normal for this device, not an error). Direct GATT, session with MaintainConnection. Transport wrapper decoded: flags/counter/fragment counters/0x0B delimiter/checksum = sum(payload)&0xFF. Strip power ON/OFF confirmed byte-for-byte against the lednetwf_ble (Home Assistant) integration references.
Oct 30	**Colour**	The legacy RGB templates (31/41/42/46) are ignored: the IOTBT family needs the v3 format `E0 01 00 A1` + hue/sat/bri. 12 hues in a row ✓. Discovery: odd hue degrees get mangled by the firmware → quantize to even degrees (1° error is invisible).
Oct 30	**Ambilight v1.0**	AmbilightOT.exe: screen capture, weighted colour average, EMA smoothing, dark scenes → white, auto-reconnect.
Oct 31	**Publication**	GitHub: PROTOCOL/AMBILIGHT/BUILDING docs, README ×5, GUI with an in-window language switch (52ea6c8), v1.1 — warm-white correction (factory "white" leans blue), v1.2 — settings autosave + power-curve saturation against the "washed out mid" of cheap LEDs.
Nov 1	**v1.3 — feedback**	The strip finally ANSWERED: query channel (0x0A family, no checksum) in FF01 → status frame `EA 81 … F0 hue sat bri` on FF02. lednotify.exe. Limit documented honestly: the strip's registers are written only by the phone app. Brightness: ladder 100→130→255 proved "byte = percent 0..100"; >100 corrupts colour.
Nov 1	**v1.4 — 20 Hz**	Removed our own "≥350 ms" clamp: sustained 16.9 cmd/s at 50 ms, burst queue 456/s with no losses, soak 3600/3600 in 213 s. GUI: the "interval" field controls everything (screen sampling = write-rate ceiling, min 50 ms).
Nov 2	**v1.5 + bug hunt**	"Run at Windows startup" checkbox (source of truth: the shortcut in shell:startup), ms-precise ambilight_log.txt, a "connection log" button and an auto-popup "LINK LOST" window on self-disconnect. In parallel the user reports the strip freezing at 50 ms after 15–30 min — under investigation (§7).

2. BLE service map of the strip



Extra service FE00: FF11 (write) · FF22 (notify) — secondary channel of the same protocol, not needed

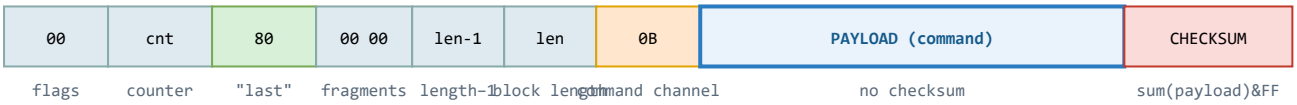
Connection algorithm (startup and reconnect)

```
1. BluetoothLEDevice.FromBluetoothAddressAsync(0x0865F0DE3701)   up to 6 tries × 5 s
2. GattSession.FromDeviceIdAsync(...) → session.MaintainConnection = true
   wait for ConnectionStatus == Connected                         up to 6 tries × 2 s
3. GetGattServicesAsync() → FFFF: FF01 (write) / FF02 (listen)
4. Write: FF01, WriteWithoutResponse; every command renders instantly
5. On link loss → full reconnect loop every ~6 s (endless)
```

> **Why no pairing:** Windows PairAsync always fails for this strip — but pairing is not needed: the GATT characteristics are open. Pairing only blocks access.

3. Protocol gramota (all verified on the real strip)

3.1 Transport wrapper: every byte in its place



Checksum formula cracked against lednetwf_ble references: sum of ALL payload bytes starting at 0x0B inclusive, modulo 256. Verified byte-for-byte.

3.2 Colour command (v3): 23 bytes that paint the colour

```
00 XX 80 00 00 0F 10 0B | E0 01 00 A1 HH SS BB 00 00 00 00 14 00 00 | CS
                        |   IOTBT family    hue sat% bri%          |  sum & 0xFF
HH = (hue << 7 | sat) >> 8 == hue/2   ·  SS = sat % (0..100)  ·  BB = bri % (0..100)
```

Command	Payload after 0x0B	Status
Power ON / OFF	`3B 23/24 00…32 00 00`	✓ verified
Colour (v3)	`E0 01 00 A1 HH SS BB …`	✓ 12 hues
White	`3B A1 00 00 64 …` (sat=0)	✓
RGB templates 31/41/42/46	`31 R G B …`	✗ strip ignores
Odd hue degree	byte_lo = sat+128	✗ firmware garbles → quantize to even
bri > 100	percent scale	✗ colour corruption

		→ ceiling 100
State query	`0B 81 8A 8B 96` (NO checksum!)	✓ v1.3
Status answer (FF02)	`EA 81 01 00 AD 0A 23 25 6E 37 F0 hue…`	✓ decoded

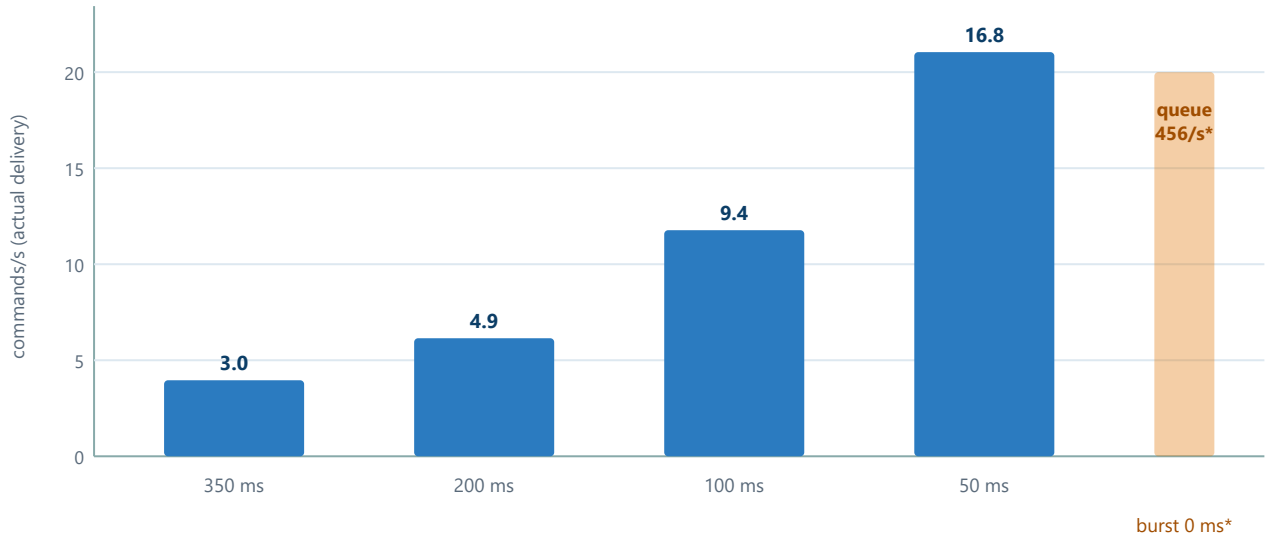
3.3 Why the "31 R G B" templates failed: this strip is a different family

From QUICK_REFERENCE: `product_id = 0x00` (IOTBT/Telink) is not one device type — **three different command families** share it. Our strip is the third family ("v3 wrapper": the E0 01 00 A1 envelope). The legacy templates belong to other families — a foreign language for this device.

Family	Power	Colour	Effects	Examples
Telink	0x71	`E2 0B hue lvl`	E0 02	IOTBT812
Segment	0x3B	`E1 03 …`	E1 01	IOTBT6BA
v3 wrapper (ours)	0x3B	`E0 01 00 A1 …`	—	ZJ-BBLA (PR #101)

4. Performance graphs

4.1 Actual delivery rate at different intervals



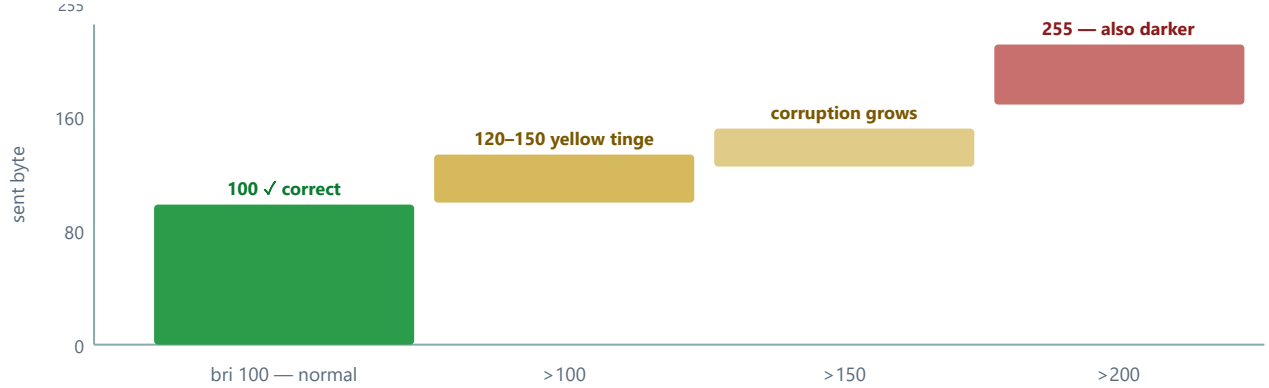
*Burst = 100 commands with no pauses: Windows queues them losslessly, but the radio only carries ~17/s — that is the physical limit of the connection. Above 17 Hz nothing gets smoother, only later.

4.2 Soak tests (both runs: 3 min and 24 min at 50 ms)

Run	Commands	Delivered	Fails	Rate	Duration	Max gap
Nov 1 · soak #1	3600	3600/3600	0	16.9/s	213 s	141 ms
Nov 2 · soak #2	24000	24000/24000	0	16.4/s	24.4 min	125 ms

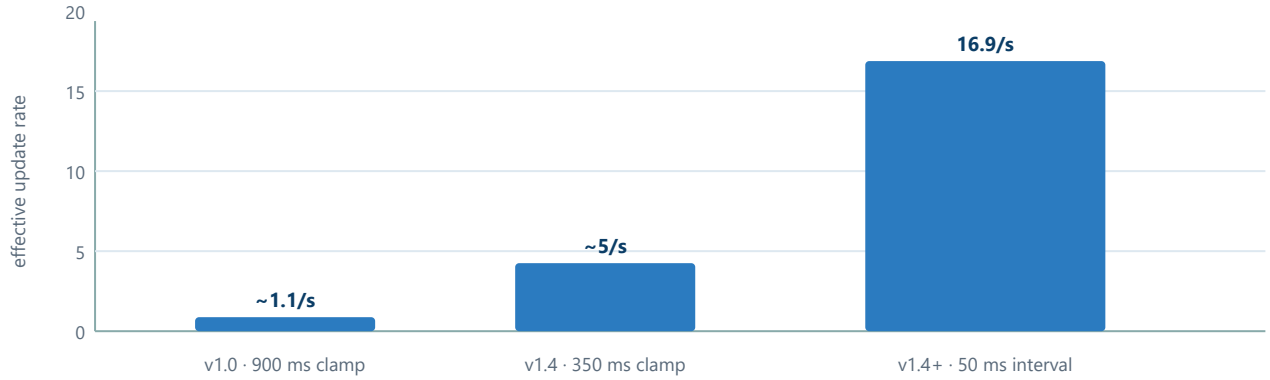
The rate never drifts → the firmware does not degrade under continuous 20 Hz load. After each run a control query + orange restore: success.

4.3 Brightness ladder (the firmware glitch above 100)



The bri byte is a PERCENT 0..100. Values above 100 are accepted but corrupt the colour (yellow-green tinge, and around 200–255 also darker) — a firmware glitch, not LED physics.

4.4 Evolution: the clamps we introduced, and how they were removed



Three "limits" were not physics but over-caution: "1–3 cmd/s", "350 ms gap needed", "brightness is 0–255". All removed by measurement: 20 Hz is safe, ceiling is 100.

5. Tooling

Tool	Purpose
AmbilightOT.exe (GUI)	The flagship: settings (interval 50 ms…3 s, sat/bri gains, min brightness, white threshold, warm-white correction), 5 interface languages, autosave, auto-reconnect; v1.5 adds the "run at Windows startup" checkbox (shell:startup shortcut), a log file and the auto-popup "LINK LOST" window.
ledctl.exe	Console: white/on/off/rgb/hsv/sweep2 — manual colour control.
lednotify.exe	Feedback and experiments: query/listen/ladder/rate/soak/send.
blescan.exe	25 s BLE scan — checks "is the strip alive in the air" (when NOBODY holds the link).
build_*.ps1	Build via csc.exe + GAC facades + WinMD — no Visual Studio at all.

6. Technical lessons (things not in the official docs)

- **Pairing is harmful and unnecessary**: Windows PairAsync always fails for this family, but GATT characteristics are open.
- **The phone is a link monopolist**: while Surplife is connected the strip is unreachable from the laptop (and vice versa). Tests only with phone BT off.
- **A connected strip does NOT advertise**: while it is in a connection it will not show up in scans. "Not in the air" often means "someone holds its link", not "it died". A death check only makes sense when nobody is connected.
- **PowerShell 5.1 is not suitable**: WinRT events (scan, notify) are not delivered (status=Started, packets=0); only writes work. Every interactive tool is C#.
- **Dynamic remote modes** (rainbow/chase) override static colour commands — set the strip to a static mode via the remote before colour tests.
- **Checksum may be non-strict**: once the strip rendered a colour despite a wrong checksum. We still rely on the verified formula.
- **The sequence byte matters**: a repeated packet counter can be dropped (from the DISCOVERIES docs) — always increment.
- **Registers ≠ our commands**: the status frame shows registers that only the phone writes. Our commands change the colour physically while the registers stay stale — feedback shows link liveness, not a mirror of our sends.

7. Hunting the strip "freeze" (Nov 2, ongoing)

> **User-reported symptom:** with the GUI at a 50 ms interval, after 15–30 minutes the strip stops updating; the GUI keeps trying to reconnect but fails; "the strip is not in the air". A power-cycle is required. The light does NOT go out — it keeps rendering the last applied colour.

Ruled out: the write rate itself

Direct reproduction failed: lednotify soak 50 ms × 24.4 min (24,000 commands, same pace as the GUI) — **0 fails**, link alive, steady 16.4/s, post-run query fine. The day before, another 3.5 min clean. So a constant 20 Hz load does not kill the firmware. A key detail from the user: the **LED part keeps living, the radio part dies** — commands are not received and the strip stops advertising.

Hypotheses (by probability)

#	Hypothesis	Reasoning	Test
1	Firmware deadlock: a rare BLE-stack hang triggered by duration / GUI-specific traffic patterns (EMA transitions, repeats) rather than sheer rate	The soak sends an extremely uniform command stream (full rainbow sweep); the GUI produces a real profile with many repeats of identical bytes	1–2 h run through the real GUI; the v1.5 log file stamps the moment
2	Windows stack/driver issue (zombie link)	"Not in the air" was probably checked from the laptop whose link went stale — the strip may have been alive; the soak shows the firmware is tough	At the moment of freeze open Surplife on the phone: sees it → PC problem; does not → firmware
3	Power/thermal: controller supply sag under long load	A power-cycle fixes it — consistent with this	Repro runs across days, note whether unload helps
4	Strip state: a dynamic mode (rainbow) fighting static commands	Similar complaints in the integration's issues	Keep the strip in static mode via the remote

The v1.5 toolset is ready for this: the log file records ms-precise timestamps and the exact LINK LOST point; the auto-popup tells the user the moment the strip drops. Next run: GUI at 50 ms for 1–2 hours, phone standing by for the live check at the moment of the freeze.

8. GitHub

README ×5 · docs/PROTOCOL.md §1–§4 · docs/AMBILIGHT.md · docs/BUILDING.md · src/*.cs · LICENSE MIT · Releases v1.1–v1.4

Release	Theme	Assets
v1.1	Warm-white correction	AmbilightOT.exe
v1.2	Autosave + saturation curve	AmbilightOT.exe
v1.3	FF02 feedback + brightness 0..100	lednotify.exe, ledctl.exe
v1.4	Fast mode: 20 Hz verified	AmbilightOT.exe, lednotify.exe
v1.5 (released)	Startup checkbox + log file + "LINK LOST" window	AmbilightOT.exe, lednotify.exe + PDF reports ×5

Commits: `e820fcc` start → `16c6009` EN docs → `dc549a0` localizations → `52ea6c8` GUI languages → `7e90b94` warm white → `6c5a10a` v1.2 → `3b9f305` MIT/screenshots → `bf46696` v1.3 → `975371a` v1.4.

9. Backlog

Task	Status	Notes

State reading (FF02)	**done (v1.3)**	query + decoding + lednotify; limitation documented.
Brightness scale 0..100	**done (v1.3)**	ceiling 100 confirmed.
20 Hz write rate	**done (v1.4)**	24.4-min soak clean; "above 17 Hz" is pointless.
Windows autostart	**done (v1.5)**	GUI checkbox + shell:startup shortcut.
Failure logging	**done (v1.5)**	file + button + "LINK LOST" window.
Strip freeze diagnosis	**in progress**	GUI run 1–2 h; phone check at the moment of the drop.
Monitor picker (list)	backlog	dropdown instead of "primary / all screens". v1.6 candidate.
Transition interpolation	backlog	2–3 micro-fade frames between colours instead of "teleports".
Hue 1° instead of 2°	research	odd degrees garbled — test first.
Brightness gamma	backlog	richer midtones in dark scenes.

10. Conclusions

- **The protocol is exhaustively decoded** — from the transport wrapper to the odd corners: the v3 family, even-degree quantization, percent brightness, the query channel, the FF02 status frame, registers. All in PROTOCOL.md.
- **Ambilight is a product, not a script**: a 5-language GUI, autosave, auto-reconnect, startup integration, a self-opening crash log. Builds without Visual Studio.
- **Every "limit" was lifted by experiment**: three of our own fears dispelled by measurement — 16.9/s sustained, ceiling 100, the 350 ms clamp unnecessary.
- **The "verify honestly" rule paid for itself**: the only open item is a rare freeze that synthetic load does not reproduce; the hunt continues through the real GUI + a phone scout at the moment of failure.

Created Nov 2, 2026 · All numbers are lednotify (rate/soak/ladder) log measurements, not theory · Source: Desktop\моды\лента\Project_Report*.html